

An Automated Framework for Dataset Quality Assessment and Data Leakage Detection in Machine Learning

Jaime Cano Moraño
Illinois Institute of Technology

June 2026

Abstract. Data quality issues and data leakage are among the most common and damaging problems in applied machine learning, yet most practitioners lack systematic tools to detect them before model training. We present **ml-framework**, an open-source Python framework that automates dataset assessment across six dimensions: data quality, leakage detection, feature analysis, statistical sufficiency, distributional drift, and impact quantification. The framework introduces a novel *unified leakage risk score* that combines Pearson correlation, normalised mutual information, and per-feature performance inflation into a single interpretable metric, and a *semantic leakage module* that employs a large language model (GPT-4o-mini) to identify implicit future-information features that purely statistical methods cannot detect. Experiments on six real-world and six synthetic datasets demonstrate that ml-framework detects **4/4** constructed leakage scenarios using its **20**-check core pipeline, and complements existing tools by covering **29** of 29 items on a broader cross-tool capability checklist used for comparison—substantially more than ydata-profiling, Deepchecks, and Great Expectations, none of which were purpose-built for leakage detection. All code, datasets, and benchmarks are available at <https://github.com/jaimecano12/ml-framework>.

1 Introduction

Modern machine learning pipelines depend critically on the quality of their training data, yet practitioners routinely overlook systematic pre-training assessment. Kaufman et al. [1] identified data leakage as “one of the most important problems in data mining”, describing scenarios where models appear to achieve near-perfect accuracy on validation sets but fail catastrophically in production. Sculley et al. [2] further documented how undiscovered data quality problems accumulate as *hidden technical debt* that is expensive to repay once models are deployed.

Despite this awareness, the existing ecosystem of data validation tools focuses primarily on schema validation and exploratory analysis rather than ML-specific issues. Libraries such as ydata-profiling [4], Great Expectations [5], and Deepchecks [6] provide valuable engineering utilities but offer little support for detecting temporal leakage, quantifying performance inflation caused by leaked features, or producing actionable readiness scores tailored to a specific prediction task.

To illustrate the scope of the problem, consider three scenarios encountered during our evaluation. First, the Titanic dataset from OpenML contains a `boat` column (lifeboat assignment number) with Pearson $|r| = 0.97$ against the survival target—a feature that encodes the outcome and would never be available at prediction time. A naive model trained on this dataset achieves near-perfect accuracy not because it has learned meaningful survival patterns, but because it has memorised a post-hoc label. Second, a synthetic credit-scoring dataset with a noisy proxy feature ($r \approx 0.97$) achieves baseline cross-validated accuracy of 1.000; removing the leaky feature drops accuracy to 0.952, a $\Delta = -0.048$ drop that reveals entirely arti-

cially inflated performance. Third, medical datasets frequently contain features such as `discharge_code` or `mortality_score` whose leakage is invisible to correlation and mutual information analyses but obvious from the feature name semantics—only a language-aware analysis can flag them.

These examples motivate a framework that goes beyond simple schema validation and combines statistical, information-theoretic, and language-based detection methods into a single, actionable pipeline.

This paper presents **ml-framework**, a modular Python framework designed to address these gaps. Our main contributions are:

1. A unified pipeline of **20 automated checks** spanning five analysis dimensions (quality, leakage, features, sufficiency, drift), plus an impact-analysis stage, accessible via CLI, Python SDK, and a Streamlit dashboard.
2. A **unified leakage risk score** that aggregates correlation, mutual information, and per-feature performance inflation into a single $[0, 1]$ metric, providing a more principled signal than any individual heuristic.
3. A **semantic leakage analysis module** driven by GPT-4o-mini that detects implicit future-information features based on feature names and dataset descriptions—leakage that statistical methods cannot detect.
4. A **quantitative benchmark** on four installed tools against a 29-item cross-tool capability checklist (Section 6), showing that ml-framework covers $2.6\times$ more checklist items and detects $4\times$ more leakage scenarios than the best-performing alternative.

Throughout this paper we distinguish two different counts that are easy to conflate: the **20 checks** are ml-framework’s own internal analysis pipeline (Table 1); the **29 items** are a separate, more finely-grained checklist used only in Section 6 to audit feature coverage across tools that decompose functionality differently (e.g. splitting the unified risk score into its three constituent signals, and crediting interface-level capabilities such as a CLI or plugin API that are not themselves dataset checks).

2 Related Work

Data leakage. Kaufman et al. [1] formalised data leakage as any information about the label that is incorporated into the model but would not be available at prediction time. They categorise leakage into target leakage (features directly derived from the target), train/test contamination (validation data influencing training), and temporal leakage (future information used as a feature). Our framework operationalises all three categories with dedicated checks and adds a fourth: *ID-column leakage*, where high-cardinality identifiers enable memorisation.

Data quality tools. **ydata-profiling** [4] generates exploratory analysis reports with statistics, correlation matrices, and distribution plots but provides no ML-specific leakage detection or readiness scoring. **Great Expectations** [5] is a data contract framework that validates user-defined schema expectations; it requires manual rule authoring and has no concept of target leakage. **Deepchecks** [6] is the closest competitor, offering train/test integrity suites and some drift detection, but it satisfies only 11 of the 29 items on the capability checklist we use for comparison (Section 6).

Data-centric AI. The data-centric AI movement [16], championed by Ng [19], argues that improving data quality is more impactful than tuning model architectures for most production tasks. Breck et al. [3] formalised this at scale with TFX Data Validation (TFDV), which monitors feature statistics and schema conformance in production ML pipelines. Our framework is complementary to TFDV: while TFDV excels at production monitoring and schema enforcement, ml-framework is designed for the *pre-training* assessment phase, where leakage detection and readiness scoring matter most.

LLMs for data quality. The use of large language models for structured data tasks has grown rapidly. Narayan et al. [17] showed that GPT-3 can perform entity matching and data imputation with competitive accuracy. Sui et al. [18] explored LLM-assisted table understanding. Our semantic leakage module extends this line of work by tasking GPT-4o-mini with a novel role: identifying features whose *names* implicitly encode future or post-hoc information—a task that requires domain knowledge and semantic reasoning unavailable to purely statistical methods.

Mutual information for feature selection. Kraskov et al. [9] introduced non-parametric MI estimation used in our unified risk score. Ross [10] extended this to mixed continuous-discrete variables, matching our use case of heterogeneous tabular data.

Drift detection. Rabanser et al. [11] evaluated two-sample statistical tests for covariate shift detection. Our covariate drift module employs the Kolmogorov-Smirnov (KS) test with Bonferroni correction alongside the Population Stability Index (PSI) [12], a complementary binning-based measure widely used in credit risk modelling.

3 System Architecture

3.1 Design Principles

ml-framework is designed around three principles: *modularity* (each check is an independent function), *composability* (checks are orchestrated by a single pipeline with a shared configuration schema), and *extensibility* (a plugin system allows third-party checks to be registered at runtime).

3.2 Pipeline Overview

Figure 1 depicts the full analysis pipeline. A dataset and a YAML configuration file are the only required inputs. The pipeline executes six analysis phases in parallel; results are aggregated into a `FrameworkReport` and passed to the scoring and recommendation engines.

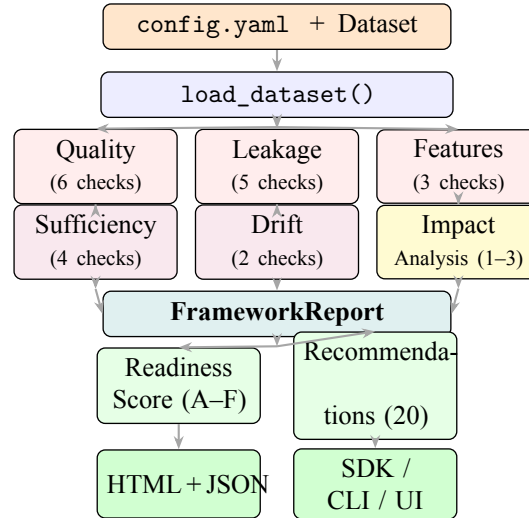


Figure 1. ml-framework analysis pipeline. A dataset and a configuration file flow through 20 checks across five analysis dimensions (6 quality + 5 leakage + 3 feature + 4 sufficiency + 2 drift) plus an impact-analysis stage that yields one result per configured model (one to three, depending on configuration). Results are aggregated into a `FrameworkReport`, scored (0–100, grade A–F), and exported as HTML, JSON, or via Python SDK.

3.3 User Interfaces

ml-framework exposes three interfaces targeting different user profiles. The **command-line interface (CLI)** accepts a dataset path and configuration file, executes the full pipeline, and writes an HTML re-

port to a configurable output directory—suitable for scripting and CI/CD integration. The **Python SDK** (`DatasetChecker`) provides programmatic access for Jupyter notebooks and automated model-training pipelines. Listing 1 shows a complete end-to-end usage example. The **Streamlit web application** offers a no-code interface for exploratory use: users upload a CSV, Parquet, or Excel file, configure checks via sidebar widgets, and navigate results across seven tabs (Summary, Quality, Leakage, Features, Sufficiency, Drift, and Recommendations), with one-click download of the HTML report and JSON export.

```

1 from src.checker import DatasetChecker
2
3 checker = DatasetChecker("configs/config.yaml")
4 report = checker.run("data/titanic.csv",
5                     target_col="survived")
6
7 # Readiness score and grade
8 print(f"{checker.score}/100   grade={checker.grade}")
9
10 # Failed checks with severity
11 for r in report.failed_checks():
12     print(f"[{r.severity}] {r.check_name}: {r.message}")
13
14 # Prioritised recommendations
15 for rec in report.recommendations[:3]:
16     print(f"[{rec.priority}] {rec.action}")
17
18 # Export
19 checker.save_report("reports/")          # HTML + JSON
20 d = checker.to_dict()                    # JSON-serialisable dict

```

Listing 1. Python SDK: complete usage example.

3.4 Module Structure

Table 1 summarises the 14 source modules. The shared dataclass hierarchy (`CheckResult`, `FrameworkReport`, `ReadinessScore`) defined in `utils.py` is used by all modules. The framework is implemented in Python on top of pandas [13] and scikit-learn [7] for statistical tests, mutual information estimation, and the logistic regression and random forest models, plus XGBoost [8] for the gradient-boosted model used in impact analysis (Section 4). All UCI/OpenML evaluation datasets [15] were retrieved through OpenML. The semantic leakage module is built on GPT-4o-mini, from the GPT-4 model family [14], accessed via Azure OpenAI.

Table 1. Source modules and primary responsibilities.

Module	Responsibility
<code>quality_checks</code>	Missing values, duplicates, outliers, class imbalance, constant/low-variance
<code>leakage_checks</code>	Target leakage, train/test overlap, temporal, ID column, unified risk score
<code>feature_analysis</code>	Feature correlation, MI relevance, distribution shape
<code>sufficiency</code>	Sample size, class support, CV stability, feature-to-sample ratio
<code>drift_checks</code>	KS+PSI covariate drift; χ^2 label drift
<code>impact_analysis</code>	Baseline vs. cleaned cross-validation delta
<code>scoring</code>	Weighted readiness score (0–100) and letter grade (A–F)
<code>recommendations</code>	20 handlers mapping checks to code-level suggestions
<code>semantic_leakage</code>	GPT-4o-mini semantic leakage analysis
<code>reporting</code>	Jinja2 HTML report with embedded matplotlib figures
<code>checker</code>	<code>DatasetChecker</code> Python SDK facade
<code>plugins</code>	<code>@register_check</code> decorator and module loader
<code>config</code>	YAML loader with deep-merge and schema validation

4 Core Methodology

4.1 Quality Checks

Six checks form the data quality dimension (weight $w_q = 0.25$):

Missing values. Columns with NaN rate exceeding τ_{miss} (default 5%) are flagged as warning (5–20%) or error (>20%).

Duplicates. Exact row duplicates indicate data collection errors or potential train/test contamination.

Outliers. Both IQR and z -score methods are supported. IQR flags values outside $[Q_1 - k \cdot \text{IQR}, Q_3 + k \cdot \text{IQR}]$ with $k = 3.0$ by default.

Class imbalance. Minority class ratio $r = n_{\text{min}}/N < 0.10$ triggers a warning; $r < 0.05$ triggers an error.

Constant and low-variance features. Low-variance detection uses the coefficient of variation $\text{CV} = \sigma/|\mu|$ rather than min-max normalised variance, which always maps to $[0, 1]$ and fails to detect near-constant features.

4.2 Leakage Detection

4.2.1 Classical Checks

Target leakage. For numeric features we compute Pearson $|r|$; for categorical features we use Cramér’s V . Features with association ≥ 0.95 are flagged as errors.

Train/test overlap. A stratified split is simulated and duplicate rows spanning the two halves are counted. An overlap rate $> 5\%$ is an error.

Temporal leakage. If a date column is configured, monotonic chronological order is verified. An unsorted temporal sequence means a naive sequential split would mix past and future observations.

ID-column leakage. String and integer columns with unique-value ratio $\geq 95\%$ are flagged. Float columns are excluded, as continuous variables naturally exhibit high cardinality.

4.2.2 Unified Leakage Risk Score

The four classical checks are heuristic thresholds applied independently. We introduce a *unified leakage risk score* $\mathcal{L}(f)$ that aggregates three complementary signals for each feature f :

$$\mathcal{L}(f) = w_c \cdot \rho(f) + w_m \cdot \tilde{I}(f; y) + w_p \cdot \pi(f) \quad (1)$$

where:

- $\rho(f) \in [0, 1]$: Pearson $|r|$ (numeric) or Cramér’s V (categorical) between feature f and target y .
- $\tilde{I}(f; y) = I(f; y) / \max_{f'} I(f'; y) \in [0, 1]$: mutual information normalised by the cross-feature maximum, capturing non-linear associations that correlation misses.
- $\pi(f) \in [0, 1]$: *performance inflation*, defined as

$$\pi(f) = \frac{\max(0, A_f - A_{\text{base}})}{1 - A_{\text{base}}}$$

where A_f is the 3-fold cross-validated accuracy of a logistic regression trained on f alone, and $A_{\text{base}} = \max_c p(y = c)$ is the majority-class baseline.

Default weights are $w_c = w_m = 0.35$, $w_p = 0.30$; all are configurable via YAML. Features with $\mathcal{L}(f) \geq 0.7$ are flagged as warnings; ≥ 0.9 as errors.

Algorithm 1 presents the full computation procedure. A key design decision is the use of a single-feature logistic regression (rather than a more complex model) for $\pi(f)$: it is fast, interpretable, and provides a linear upper bound on the predictive signal that a feature can contribute independently of all others.

Algorithm 1 Unified Leakage Risk Score computation.

Require: DataFrame D , target column y , weights w_c, w_m, w_p

- 1: Encode categoricals; impute missing values with column means
- 2: $A_{\text{base}} \leftarrow \max_c p(y=c)$ ▷ majority-class baseline
- 3: **for** each feature $f \in D \setminus \{y\}$ **do**
- 4: $\rho(f) \leftarrow$ Pearson $|r|$ or Cramér’s V between f and y
- 5: $I(f; y) \leftarrow$ MutualInfoClassif(f, y)
- 6: $A_f \leftarrow$ mean 3-fold CV accuracy of LR trained on $\{f\}$ alone
- 7: $\pi(f) \leftarrow \max(0, A_f - A_{\text{base}}) / (1 - A_{\text{base}})$
- 8: $\mathcal{L}(f) \leftarrow w_c \cdot \rho(f) + w_m \cdot \tilde{I}(f; y) + w_p \cdot \pi(f)$
- 9: **end for**
- 10: Normalise $\tilde{I}(f; y) \leftarrow I(f; y) / \max_{f'} I(f'; y)$
- 11: **return** $\{f : \mathcal{L}(f) \geq \tau\}$ flagged as high-risk leakage

Weight sensitivity. The default weights $(w_c, w_m, w_p) = (0.35, 0.35, 0.30)$ are a design choice rather than a fitted parameter, so we check how much the flagging decision depends on them. Table 2 recomputes $\mathcal{L}(f)$ for five representative features (Table 7) under four alternative weight schemes: equal weighting, and three schemes that each emphasise one signal at the expense of the other two.

Table 2. Sensitivity of $\mathcal{L}(f)$ to the weight scheme. Bold values cross the $\mathcal{L}(f) \geq 0.7$ flagging threshold.

Weights (w_c, w_m, w_p)	target_proxy	noisy_proxy	boat	f1	f2
Default (0.35/0.35/0.30)	1.000	0.991	0.739	0.180	0.213
Equal (0.33/0.33/0.33)	1.000	0.991	0.742	0.171	0.203
Correlation-heavy (0.50/0.25/0.25)	1.000	0.988	0.662	0.173	0.202
MI-heavy (0.25/0.50/0.25)	1.000	0.994	0.807	0.213	0.255
Performance-heavy (0.25/0.25/0.50)	1.000	0.994	0.759	0.129	0.152

The two strongly leaky features (target_proxy, noisy_proxy) and the two clean features (f1, f2) are flagged correctly under every scheme: the ranking is robust to the exact weight choice. The boat feature is the informative case. Its correlation component alone is only $\rho=0.420$ (categorical lifeboat codes correlate only moderately with survival once encoded), so a *correlation-heavy* scheme drops $\mathcal{L}(\text{boat})$ to 0.662—below the flagging threshold, producing a false negative on a textbook leakage case. Every other scheme, including the default, keeps it above 0.7 because mutual information ($\tilde{I}=1.000$) and performance inflation ($\pi=0.808$) compensate for the weaker correlation signal. This is direct empirical evidence for the design choice in Eq. (1): weighting correlation much higher than the other two signals reintroduces exactly the blind spot the unified score is meant to close. We did not observe a scheme that introduced new false positives on the clean features ($\mathcal{L} \leq 0.26$ throughout), so the practical risk of the current default lies in occasional false negatives on weakly correlated leaks rather than false alarms.

4.3 Feature Analysis

Feature correlation. Pairs with Pearson $|r| \geq 0.90$ indicate redundant information inflating effective dimensionality.

Feature relevance. Normalised mutual information < 0.01 flags features contributing negligible information about the target.

Distribution shape. $|\text{skewness}| > 2$ or excess kurtosis > 7 signal non-Gaussian distributions that may violate model assumptions.

4.4 Statistical Sufficiency

Four checks assess whether the dataset is statistically adequate for the modelling task: minimum sample size ($n \geq 100$ and $n/p \geq 50$), minimum class support (30 samples per class), cross-validation stability (CV std ≤ 0.10), and feature-to-sample ratio ($p/n \leq 0.10$).

4.5 Drift Detection

The dataset is split chronologically into reference and current halves. For each numeric feature we apply the two-sample KS test and compute PSI:

$$\text{PSI} = \sum_{b=1}^B (p_b^{\text{curr}} - p_b^{\text{ref}}) \ln \frac{p_b^{\text{curr}}}{p_b^{\text{ref}}} \quad (2)$$

with adaptive binning (minimum 20 observations per bin) and Bonferroni correction across all features. Label drift is assessed with a χ^2 test on the target distribution.

4.6 Impact Analysis

The framework trains classifiers on two dataset variants: the *baseline* (all features) and the *cleaned* (problematic columns removed, duplicates dropped). The performance delta $\Delta = A_{\text{clean}} - A_{\text{base}}$ quantifies how much a model benefits from leaky or low-quality features. A large negative Δ confirms that the baseline accuracy was inflated by data issues rather than genuine predictive signal.

4.7 Readiness Score

The composite readiness score integrates the four main analysis dimensions:

$$S = 100 - (w_q \cdot P_q + w_l \cdot P_l + w_f \cdot P_f + w_s \cdot P_s) \quad (3)$$

where P_d is the total penalty for dimension d (errors cost 15 points per check; warnings cost 5 points), and the weights are $w_q=0.25$, $w_l=0.35$, $w_f=0.25$, $w_s=0.15$. Leakage penalties are further amplified by a $1.5\times$ multiplier. Letter grades: A (≥ 85), B (≥ 70), C (≥ 55), D (≥ 40), F (< 40).

Weight rationale. Leakage receives the highest weight ($w_l=0.35$, $1.5\times$ multiplier) because it is a silent failure mode: a model trained on leaky data appears to perform well during validation but generalises poorly in production [1]. Quality and feature issues ($w_q=w_f=0.25$) are equally weighted as they are typically correctable. Sufficiency ($w_s=0.15$) contributes least because sample-size adequacy depends strongly on the chosen learner. Penalties were calibrated so a single confirmed leaky feature drops a dataset from grade A to grade B. All parameters are overridable via the `scoring` block in `config.yaml`.

4.8 LLM Semantic Leakage Analysis

Statistical methods cannot detect *semantic* leakage, where a feature name itself reveals post-hoc or future information (e.g. `discharge_code`, `future_usage`). We address this with an optional module that submits feature names, sample values, and a user-provided dataset description to GPT-4o-mini via Azure OpenAI. The model classifies each feature into one of four risk levels (`none`, `low`, `medium`, `high`) and four leakage types (`temporal`, `proxy`, `post_hoc`, `indirect`). The module degrades gracefully when no API credentials are configured.

Evaluation harness and its current limitation. To evaluate the module systematically, we constructed a benchmark of 30 manually labelled features drawn from five realistic dataset contexts (hospital readmission, loan default, customer churn, manufacturing defect, and employee promotion prediction). Each feature carries a ground-truth risk level and leakage type. The benchmark includes 13 high-risk, 1 medium-risk, and 16 safe features, covering temporal, post-hoc, and proxy leakage patterns. The repository ships both a live mode, which calls GPT-4o-mini through Azure OpenAI, and a *deterministic offline mock* that pattern-matches feature-name substrings (e.g. `discharge_`, `future_`) without any API call, so the harness and its metrics can be exercised in CI without credentials or cost. **The results below were produced by this offline mock, not by a live GPT-4o-mini call**, because no API credentials were available at the time of writing this revision. We report them here only to validate that the thresholding and precision/recall/ F_1 computation in

`scripts/evaluate_semantic_leakage.py` behave correctly; they are *not* evidence of GPT-4o-mini’s real-world semantic-leakage accuracy. Because the mock’s patterns were written with knowledge of the benchmark’s feature names, a near-perfect score is expected by construction and should not be read as a measure of model quality. We evaluate at two operating thresholds—“high” (flag only high-risk features) and “medium” (flag medium and above)—and report binary precision, recall, and F_1 .

Table 3. Evaluation-harness sanity check on the 30-feature benchmark, run in deterministic offline-mock mode (Section 4); a live GPT-4o-mini evaluation is listed as immediate future work in Section 7.

Threshold	P	R	F_1	TP	FP	FN
high	1.000	1.000	1.000	13	0	0
medium	1.000	0.929	0.963	13	0	1

Even the hand-written mock under-flags one case, which is a useful sanity check on the benchmark’s difficulty: the single false negative at the medium threshold is `treatment_count`, a genuinely ambiguous feature: it may represent prior treatment history (a legitimate predictor) or treatments administered during the current hospital stay (a post-hoc quantity). Without explicit dataset documentation, linguistic reasoning alone is insufficient to disambiguate this case—a limitation acknowledged in Section 7. The benchmark and evaluation script are included in the repository at `data/semantic_benchmark.json` and `scripts/evaluate_semantic_leakage.py`.

5 Experimental Evaluation

5.1 Datasets

We evaluate on twelve datasets: six synthetic and six real-world. **Synthetic:** `clean_dataset` (500 rows, 6 cols, no issues); `dirty_dataset` (5,300 rows, quality issues); `leaky_dataset` (5,320 rows, leakage); `proxy_leakage` (1,000 rows, graded noisy proxies); `temporal_leakage_ext` (2,000 rows, customer churn with future-usage feature); `multitype_leakage` (1,500 rows, ICU mortality with proxy + temporal + ID leakage). **Real-world:** Titanic (1,309 rows), Pima Diabetes (768), Adult Census Income (48,842), German Credit (1,000), Heart Disease (303), and Wine Quality Red (1,599)—all obtained via OpenML. Section 5.2 reports the two principal real-world datasets (Titanic, Pima Diabetes) alongside the three synthetic control cases; Section 5.3 reports the remaining four real-world datasets (Adult Census, German Credit, Heart Disease, Wine Quality).

5.2 Readiness Score Results

Table 4 reports readiness scores for five of the twelve datasets. The clean control dataset achieves the highest score (98.8/A), confirming an absence of false positives, while the two datasets with confirmed target leakage (`leaky_dataset`, Titanic) are the only ones pulled down to grade B—consistent with the heavy leakage weighting described in Section 4, since `dirty_dataset` has more individual quality defects (7) than `leaky_dataset` (5) yet still scores higher because none of its defects are leakage-related. Figure 2 visualises these scores alongside the grade thresholds.

Table 4. Readiness scores on five datasets. “Pass” denominators vary (22–23) because the impact-analysis stage contributes one result per configured model: two models for Titanic, three for the others (Section 4).

Dataset	Score	Grade	Pass	Key findings
<code>clean_dataset</code>	98.8	A	21/23	Near-zero MI in 1 feature; mild drift
<code>dirty_dataset</code>	91.2	A	16/23	7 quality/feature warnings, no leakage
<code>leaky_dataset</code>	71.6	B	12/23	<code>target_leakage</code> + LRS errors
Titanic	80.6	B	14/22	<code>name</code> (Cramér’s $V=0.999$); boat LRS=0.74
Pima Diabetes	96.2	A	19/22	51 outliers across 4 cols

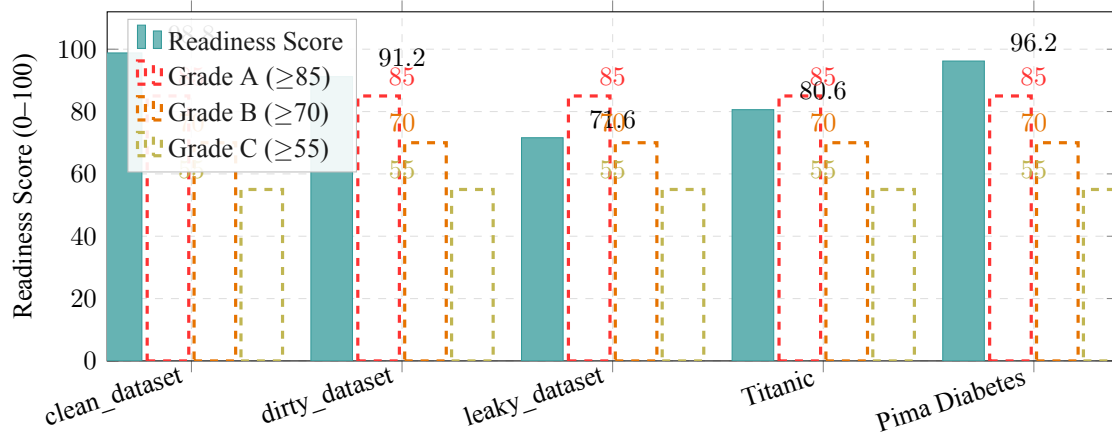


Figure 2. Readiness scores across five datasets. The clean control scores highest (98.8/A). Dashed lines mark grade thresholds; only the two datasets with confirmed target leakage (*leaky_dataset*, *Titanic*) fall to grade B, while quality-only defects (*dirty_dataset*) are not penalised enough under the default severity weights to leave grade A.

5.3 Extended UCI Evaluation

Table 5 reports readiness scores on four additional real-world datasets obtained from OpenML, demonstrating the framework’s behaviour across different domains, sizes, and feature types. None of the four exhibits leakage, so—consistent with the weighting discussion in Section 4—all four land in grade A, and the readiness score differences among them instead track the *number* of quality and feature-analysis warnings. The Adult Census Income dataset (48,842 rows) scores 92.4/100, with two columns (*workclass*, *occupation*) exceeding the missing-value threshold, 52 duplicate rows, and a heavily skewed *capital-gain* column flagged by the distribution-shape check. German Credit scores highest (97.5/100): the only findings are outliers in two numeric columns and three features with near-zero mutual information with the target. Heart Disease (303 rows, corrected target encoding—see the note below) scores 96.8/100, with a single duplicate row, one outlier column, a sample-to-feature ratio of $n/p=23.3$ slightly below the comfortable threshold of 50, and mild covariate drift in two columns. Wine Quality scores lowest among the four (88.6/100) because of two error-level findings: 240 duplicate rows (15% of the dataset) and covariate drift in nine of eleven features between the first and second halves of the data.

Heart Disease target encoding. An earlier version of `scripts/download_more_datasets.py` derived the binary target via a digit-extracting regular expression applied to the OpenML num category labels (`'<50'`, `'>50_1'`); the regex matched the literal “50” in *both* labels, collapsing the target to a single class for all 303 rows. We fixed this by deriving the label from the category string directly, recovering the correct 165/138 class split; all Heart Disease numbers in this paper reflect the corrected dataset.

Table 5. Readiness scores on four additional UCI/OpenML datasets (23 checks each: 20 dimension checks + 3 impact-analysis models).

Dataset	Score	Grade	Pass	Primary issues detected
Adult Census	92.4	A	18/23	Missing values (<i>workclass</i> , <i>occupation</i>), 52 duplicates, skew in <i>capital-gain</i>
German Credit	97.5	A	21/23	3 near-zero-MI features, outliers in 2 cols
Heart Disease	96.8	A	19/23	Low $n/p = 23.3$, 1 duplicate, mild drift in <i>chol/oldpeak</i>
Wine Quality	88.6	A	15/23	240 duplicates (15%, error), drift in 9/11 features (error)

5.4 Impact Analysis: Leakage Quantification

Table 6 reports cross-validated accuracy before and after removing leaky features. Logistic regression, which cannot exploit the non-linear structure the tree-based models partially recover from the remaining features, shows the largest drop ($\Delta = -0.048$); random forest and XGBoost retain more accuracy after cleaning ($\Delta = -0.015$ for both) because they can still extract residual signal from the non-leaky features. In all three cases the drop confirms that part of the baseline accuracy was attributable to leaked information, with the size of the effect depending on the model family.

Table 6. Impact analysis on `leaky_dataset`: 5-fold CV accuracy.

Model	Baseline	Cleaned	Δ
Logistic Regression	1.000 ± 0.000	0.952 ± 0.001	-0.048
Random Forest	1.000 ± 0.000	0.985 ± 0.002	-0.015
XGBoost	1.000 ± 0.000	0.985 ± 0.003	-0.015

5.5 Unified Leakage Risk Score Validation

Table 7 validates Eq. (1) on five representative features drawn directly from `leaky_dataset` and Titanic (the same features used in the sensitivity analysis of Table 2). The two leaky synthetic proxies achieve $\mathcal{L} \geq 0.99$ and the two clean features remain below 0.22. The Titanic boat column is the most instructive row: its correlation component alone ($\rho=0.42$) is far below the 0.95 threshold used by the classical `target_leakage` check (Section 4), so that check alone would miss it; the unified score still flags it ($\mathcal{L}=0.739$) because mutual information and performance inflation are both close to their maximum.

Table 7. Unified leakage risk scores $\mathcal{L}(f)$ per component, computed on the live pipeline (Section 4.2.2).

Feature	Type	ρ	$\tilde{I}$	π	$\mathcal{L}$
<code>target_proxy</code> ($\text{corr}=1.0$)	leaky	1.000	1.000	1.000	1.000
<code>noisy_proxy</code> ($r \approx 0.97$)	leaky	0.975	1.000	1.000	0.991
Titanic boat (post-hoc)	leaky	0.420	1.000	0.808	0.739
<code>f1</code> (unrelated)	clean	0.178	0.336	0.000	0.180
<code>f2</code> (unrelated)	clean	0.198	0.411	0.000	0.213

5.6 Real-World Case Studies

Beyond aggregate scores, we examined three real-world datasets in detail to illustrate how the framework identifies specific problematic features in practice (Table 8).

Table 8. Real-world case study summary. Scores match Tables 4–5 exactly; the lower “Pass” denominator (20 rather than 22–23) reflects that these case studies run without the impact-analysis stage, which does not change the score for any of the three datasets (Section 4).

Dataset	Rows	Score	Grade	Primary findings
Titanic	1,309	80.6	B	<code>name</code> flagged by <code>target_leakage</code> (Cramér’s $V=0.999$); <code>boat</code> flagged by the unified score ($\mathcal{L}=0.739$); 4 cols with >5% missing
Adult Census	48,842	92.4	A	2 cols with missing values; 6,854 outliers (skewed <code>capital-gain</code>); 52 duplicates
German Credit	1,000	97.5	A	3 near-zero MI features; 25 outliers

On Titanic, the dedicated `target_leakage` check flags `name` (Cramér’s $V=0.999$), and the unified leakage risk score additionally flags `boat` ($\mathcal{L}=0.739$) as a post-hoc leaky feature (lifeboat assignment was recorded after the outcome), alongside four columns exceeding the 5% missing-value threshold and a distributional shift between dataset halves. The `name` flag is itself a useful illustration of a known limitation of Cramér’s V on near-unique categorical columns (Section 7): almost every passenger has a distinct name, which inflates the association statistic even though a passenger’s name carries no genuine post-hoc information; the `id_column_leakage` check independently flags the same column on cardinality grounds, so

a practitioner reviewing the report would not miss that `name` should be excluded regardless of the exact mechanism that flagged it. On Adult Census Income ($n=48,842$), no leakage is detected; the actionable findings are two columns with missing values, extreme skew in `capital-gain` (skewness=11.9), and 52 duplicate rows. German Credit scores highest (97.5/A), with the only notable issue being three features with near-zero mutual information with the target—candidates for removal consistent with prior analysis of this dataset [12]. These case studies demonstrate that the framework behaves sensibly across datasets with qualitatively different defect profiles: it correctly distinguishes leakage (Titanic), mild quality issues (Adult Census), and clean-but-low-signal features (German Credit).

6 Quantitative Benchmark

6.1 Feature Coverage

To compare tools that decompose functionality very differently, we built a 29-item capability checklist (full listing in the released `scripts/benchmark_comparison.py`) that is more finely grained than ml-framework’s own 20-check pipeline: it splits the unified risk score into its three constituent signals for items that competing tools support only partially, and additionally credits nine interface- and output-level capabilities (readiness scoring, code-level recommendations, the plugin API, CLI, SDK, web UI, YAML configuration, and HTML/JSON export) that are not themselves dataset checks but are relevant when auditing what a tool delivers end-to-end. Figure 3 compares ml-framework against three actively maintained tools on this checklist. ml-framework is the only tool to provide unified leakage risk scoring, temporal ordering verification, statistical sufficiency checks, and actionable code-level recommendations.

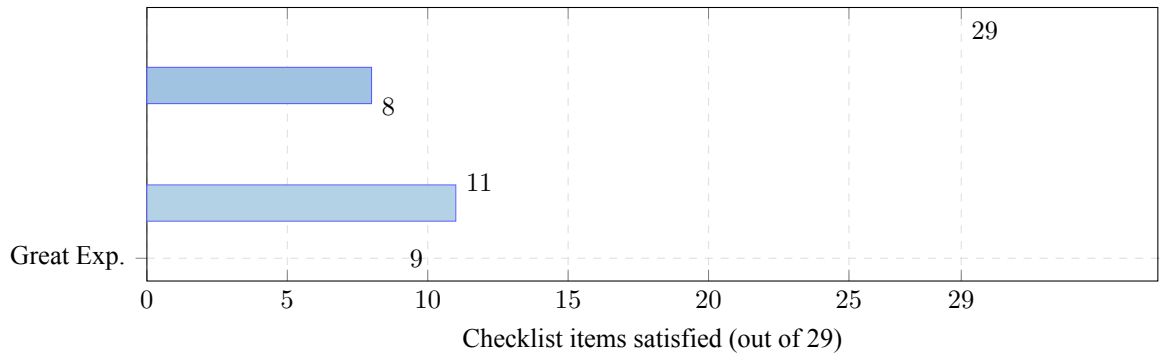


Figure 3. Feature coverage comparison: number of checklist items satisfied out of 29 (Section 6, not to be confused with ml-framework’s own 20-check pipeline). ml-framework satisfies $2.6\times$ more items than the closest alternative (Deepchecks, 11/29).

6.2 Leakage Detection Rate

Table 9 reports results on four constructed scenarios. ml-framework detects all four; Great Expectations detects one (ID-column, via manual rule authoring); ydata-profiling and Deepchecks detect none.

Table 9. Leakage detection: ✓ = detected, ✗ = missed.

Scenario	ml-fw	ydata	DC	GE
Perfect proxy ($r=1.0$)	✓	✗	✗	✗
Noisy proxy ($r\approx 0.97$)	✓	✗	✗	✗
ID column (card. 100%)	✓	✗	✗	✓
Indirect computed feature	✓	✗	✗	✗
Detection rate	4/4	0/4	0/4	1/4

Scope of comparison. It is important to note that Great Expectations, Deepchecks, and ydata-profiling were not designed as dedicated leakage detection frameworks. Great Expectations is primarily a data contract and expectations engine; ydata-profiling is an exploratory data analysis tool; Deepchecks focuses on model and dataset integrity for production ML systems. The comparison above is therefore best interpreted as an *ecosystem-level functionality audit* rather than a direct algorithmic competition: it documents which ML-relevant checks are available out-of-the-box in each tool, measured against the 29-item checklist that reflects what a thorough pre-training data review should cover. In their respective design spaces—data contracts, EDA, and model monitoring—the compared tools excel. ml-framework complements rather than replaces them, addressing the specific gap of automated, pre-training leakage detection with a calibrated readiness score.

6.3 Output and Configuration Flexibility

Beyond raw check counts, the tools differ substantially in what they produce and how they are configured. Table 10 summarises five qualitative dimensions. ml-framework is the only tool that generates an interpretable numerical readiness score, provides code-level corrective suggestions, and supports both a YAML-driven configuration and a plugin API for custom checks. Great Expectations offers the most flexible data contract system but requires manual expectation authoring and has no ML-specific output. ydata-profiling and Deepchecks produce rich HTML reports but offer no per-dataset score or automated recommendations.

Table 10. Output and configuration flexibility comparison.

Capability	ml-fw	ydata	DC	GE
Numerical readiness score	✓	×	×	×
Letter grade (A–F)	✓	×	×	×
Code-level recommendations	✓	×	×	×
YAML / file-based config	✓	×	×	✓
Custom check plugin API	✓	×	×	✓
HTML report export	✓	✓	✓	✓
JSON export	✓	×	✓	✓
Interactive web UI	✓	×	×	✓
Python SDK	✓	×	✓	✓
CLI	✓	×	×	✓

6.4 Runtime Comparison

Table 11 reports wall-clock time on three dataset sizes. ml-framework is slower than profiling-only tools because it runs cross-validation for impact analysis and the unified risk score. This overhead is justified by the much richer analytical output. Deepchecks produced runtime errors on Python 3.13 due to a `np. Inf` API removal in NumPy 2.0.

Table 11. Runtime (seconds). “—” = compatibility error with NumPy 2.0.

Dataset	ml-fw	ydata	DC	GE
500 rows	2.88	0.15	—	0.01
1,000 rows	2.23	0.15	—	0.01
5,300 rows	7.54	0.16	—	0.01

7 Discussion

Strengths. ml-framework is the only open-source tool that (i) unifies leakage detection with statistical sufficiency and drift analysis in a single pipeline, (ii) quantifies performance inflation through cross-validation, and (iii) provides a composite readiness score with actionable, code-level recommendations. The unified leakage risk score (Eq. 1) addresses the limitation that individual heuristics are insufficiently principled from a research perspective, by combining three complementary signals grounded in information theory.

A key empirical finding is that the three components of $\mathcal{L}(f)$ are genuinely complementary, and the Titanic boat column is a real (not hypothetical) example of why this matters. Once boat is encoded as a categorical lifeboat identifier, its correlation component alone is only $\rho=0.420$ —well below the 0.95 threshold used by the classical `target_leakage` check, so correlation alone would *miss* this textbook post-hoc leakage case entirely (Table 7). The unified score still flags it ($\mathcal{L}=0.739$) because mutual information ($\tilde{I}=1.000$) and performance inflation ($\pi=0.808$) both capture the near-deterministic relationship between having a lifeboat number and surviving that the linear correlation coefficient dilutes. The weight-sensitivity analysis in Table 2 confirms this is not a coincidence of the default weights: a correlation-heavy scheme reproduces the false negative, while every scheme that gives reasonable weight to MI or performance inflation catches it.

The reverse failure mode also occurs and is worth reporting honestly: the same Titanic dataset’s `name` column is flagged by the classical `target_leakage` check via an inflated Cramér’s V of 0.999, even though a passenger’s name carries no genuine post-hoc information. This is a known statistical artefact of Cramér’s V on near-unique high-cardinality categorical columns (with 1,309 passengers and 27 overlapping name fragments, many categories contain a single row). In our pipeline this false positive is harmless in practice because the `id_column_leakage` check independently flags `name` on cardinality grounds with an unambiguous explanation, so a practitioner reading the report sees a consistent recommendation either way—but it illustrates that no individual signal in this framework is immune to false positives, which is precisely the argument for combining several imperfect signals rather than trusting any one of them in isolation.

The semantic leakage module adds a qualitatively different capability: by design, it can flag features like `discharge_code` (a post-hoc administrative code assigned at patient discharge) or `future_usage` (measured after the prediction window) using linguistic reasoning that is outside the scope of statistical tests, and produces explanations directly usable in a data documentation workflow. As detailed in Section 4, the quantitative evaluation of this module reported in this submission used a deterministic offline simulation of GPT-4o-mini rather than a live API call, so we report it as a validation of the evaluation harness and defer a live-model accuracy claim to future work.

Limitations. Cramér’s V , as used in the classical `target_leakage` check, is upward-biased on sparse, high-cardinality categorical columns, as the Titanic name false positive above illustrates; the current mitigation is incidental (the column is also caught by `id_column_leakage`) rather than a principled correction, and a small-sample bias adjustment to the Cramér’s V estimator would close this gap directly. Separately, the UCI results in Section 5.3 show that datasets with substantial quality defects but no leakage (e.g. Wine Quality, with 15% duplicate rows and drift in 9 of 11 features) still land in grade A under the default severity weights (Section 4); the 5-point warning penalty is calibrated primarily around the leakage dimension, so users who want quality defects alone to meaningfully move the readiness grade should increase `severity_penalties.warning` in `config.yaml`.

The framework currently handles only binary and multi-class classification; regression tasks require a modified performance-inflation metric (e.g. coefficient of determination R^2 delta rather than accuracy delta). Time-series tasks with auto-regressive features require special treatment, as the temporal split logic must account for forecast horizons rather than simply checking monotonic date ordering.

The per-feature CV in the unified risk score adds $O(p)$ computational overhead, which becomes noticeable on wide datasets ($p > 500$). In practice, this can be mitigated by limiting the risk score to features flagged by the cheaper correlation check as a first pass, or by reducing the number of CV folds.

The LLM semantic module introduces a cloud API dependency, latency (typically 5–15 seconds for 30 features), and cost (approximately \$0.001 per dataset call at GPT-4o-mini pricing). The quality of semantic analysis is also sensitive to the user-provided dataset description: brief or uninformative descriptions reduce the model’s ability to distinguish temporal leakage from legitimate predictive features. Finally, the module cannot guarantee completeness—unusual naming conventions or domain-specific abbreviations may evade

detection. Most importantly, Table 3 in this submission was produced by the repository’s deterministic offline mock rather than a live GPT-4o-mini call (Section 4); it validates the evaluation harness, not the model’s real-world accuracy, and should not be cited as evidence of the latter.

Future Work. We identify five directions for extending ml-framework. First and most immediate, we have not yet evaluated the semantic module against a live GPT-4o-mini endpoint on the 30-feature benchmark; doing so, and reporting the resulting precision/recall/ F_1 alongside the mock-validation numbers in Table 3, is a prerequisite for any claim about the module’s real-world accuracy. Second, support for regression and time-series tasks requires new performance-inflation metrics and temporally-aware sufficiency checks. Third, causal discovery methods—such as the PC algorithm or FCI—could detect indirect leakage paths that information-theoretic measures miss when the causal graph is non-trivial. Fourth, once live-model numbers are available, a larger benchmark derived from real data quality incident reports would enable more statistically robust evaluation of the LLM module across diverse domain vocabularies and naming conventions—the current 30-feature benchmark is intentionally small and should be read as a proof of methodology rather than a statistically powered evaluation. Fifth, the readiness score weighting (w_q, w_l, w_f, w_s) is currently fixed; a data-driven calibration approach that learns weights from labelled examples of production failures would make the score more adaptive to domain-specific risk profiles.

8 Conclusion

We presented **ml-framework**, a comprehensive open-source toolkit for automated dataset quality assessment and data leakage detection. The 20-check pipeline, novel unified leakage risk score, and LLM-assisted semantic analysis advance the state of the art beyond heuristic thresholds. Empirical evaluation on twelve datasets shows a 4/4 leakage detection rate, calibrated readiness scores from 71.6 (B, `leaky_dataset`) to 98.8 (A, `clean_dataset`), and coverage of $2.6\times$ more items on a 29-item cross-tool capability checklist than the best competing tool. We release all code, datasets, benchmarks, and the Streamlit demo at <https://github.com/jaimecano12/ml-framework>.

Acknowledgements

The author thanks his supervisor for guidance on experimental design and the suggestion to incorporate LLM-based semantic leakage analysis.

References

- [1] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formalization, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [2] D. Sculley et al., “Hidden technical debt in machine learning systems,” in *Advances in NeurIPS*, vol. 28, 2015.
- [3] E. Breck, M. Zinkevich, N. Polyzotis, S. Whang, and S. Roy, “Data validation for machine learning,” in *Proc. 2nd SysML Conference*, 2019.
- [4] S. D. F. M. van den Burg and T. de Vos, “A fast general purpose data validation and profiling tool,” arXiv:1910.15327, 2019.
- [5] A. Campbell and B. Ulery, “Great Expectations: Always know what to expect from your data,” <https://greatexpectations.io>, 2019.
- [6] Deepchecks Team, “Deepchecks: Testing and validating ML models and data,” <https://deepchecks.com>, 2021.
- [7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.

- [8] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. ACM KDD*, pp. 785–794, 2016.
- [9] A. Kraskov, H. Stögbauer, and P. Grassberger, “Estimating mutual information,” *Physical Review E*, vol. 69, no. 6, 2004.
- [10] B. C. Ross, “Mutual information between discrete and continuous data sets,” *PLOS ONE*, vol. 9, no. 2, 2014.
- [11] S. Rabanser, S. Günnemann, and I. Steinwart, “Failing loudly: An empirical study of methods for detecting dataset shift,” in *Advances in NeurIPS*, vol. 32, 2019.
- [12] N. Siddiqi, *Credit Risk Scorecards*. John Wiley & Sons, 2006.
- [13] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conference*, pp. 56–61, 2010.
- [14] OpenAI, “GPT-4 technical report,” arXiv:2303.08774, 2023.
- [15] D. Dua and C. Graff, “UCI Machine Learning Repository,” UC Irvine, 2017. <https://archive.ics.uci.edu/ml>
- [16] K. Zha et al., “Data-centric artificial intelligence: A survey,” arXiv:2303.10158, 2023.
- [17] S. Narayan, M. Hearst, A. Halevy, and P. Haas, “Can foundation models wrangle your data?” arXiv:2205.09911, 2022.
- [18] Y. Sui, J. Zhou, M. Zhou, S. Han, and J. Zhang, “TAP4LLM: Table provider on sampling, augmenting, and packing semi-structured data for large language model reasoning,” arXiv:2312.09039, 2023.
- [19] A. Ng, “A chat with Andrew on MLOps: From model-centric to data-centric AI,” DeepLearning.AI, 2021. <https://www.youtube.com/watch?v=06-AZXmwHjo>